MANDATE — SIMULATED PROTOTYPE

Security Self-Assessment

Threat model, safeguards, measured results, and known risks

Submission · 2026 Shenzhen International FinTech Competition, International Track, Topic C

SIMULATED — NO REAL MONEY MOVES

Team: Mandate — Arif Hussain (Team Leader), Ankon Sinha, Miftahul Alam Adib
Institution: Shahjalal University of Science and Technology (SUST), Bangladesh
Repository: github.com/arif124713/Shenzhen-International-FinTech-Competition-FinTechathon-International-Track
Live demo: mandate-nu-ten.vercel.app
Generated: 2026-10-19

> Submission item 05. All results are synthetic/sandbox. Numbers come from `eval/reports/latest.json`. Enforcement points are real file:function references — verify by grepping the tree.

---

1. Scope and system description

Mandate is a web app where an AI agent helps international students with cross-border payments, but the authority to move money comes from a **user-signed mandate** and every decision is made by **deterministic code** (`packages/policy_engine`), not by an LLM. Documents are read by an **isolated reader with no tools**. An LLM may read and propose; only a signed engine decision authorizes execution.

---

2. Permission model — tiers T0–T4

Tier	Meaning	Decision	Enforcement point
T0	read-only	—	UI views only
T1	propose only	—	planner output is a proposal, never executed
T2	auto-execute within caps to verified/allowlisted payees	ALLOW	`policy_engine/decide.py:decide()` → `apps/api/services/executor.py:execute()`
T3	co-signer approval required	ESCALATE / HOLD	`apps/api/services/approvals.py` ; engine re-run with the approval as an input
T4	forbidden	DENY	`decide()` default-deny + precedence DENY > HOLD > ESCALATE > ALLOW

---

3. Trust boundaries

- **Untrusted:** uploaded documents, user free text, planner output, reader output until validated.
- **Semi-trusted (manipulable):** the LLMs (reader, compiler, planner).
- **Trusted:** policy engine, signature-verified registry, signed mandate, sandbox ledger, audit log writer.
- **Rules:** the reader has no tools (enforced by an AST import test + a runtime signature test); the planner never sees raw document text (test double verifies); only `services/executor.py` imports `rail_sim` (AST test); the engine is a pure function with no network/clock/randomness.

---

4. Threat model

Adversaries: a malicious document author; a tampered registry; a replayed/forged approval; a planner coerced into a wrong action. Entry points: document upload, natural-language rules, the proposal API, the approval token. Attack families A01–A25 (`eval/attacks/catalog.yaml`) model these.

---

5. Controls mapped to threats — test id

Control	Enforcement point	Test	Measured result
Deterministic decision + signing	`policy_engine/decide.py:decide()` , `tokens.py:issue_decision_token()`	U-ENG-*, P-I2	all pass
Execution verification (signature, ALLOW, expiry, nonce, `action_hash` , mandate/registry validity)	`services/executor.py:execute()`	U-EXEC-01…04	all pass
Single execution path	AST test	U-EXEC-05	passes
Reader isolation	`agents/reader/__init__.py:read_document(document, complete)`	U-EXEC-06	passes
Registry check	`policy_engine/decide.py:_check_registry()`	U-ENG-05, U-REG-01	passes
Approval binding + single use	`services/approvals.py` , engine re-run	U-APPR, U-EXEC-03	passes
Audit chain + checkpoints	`services/audit_log.py:append()/checkpoint()`	U-AUDIT, U-AUDIT-02	passes
Engine purity	AST import scan	U-ENG-14, P-I3	passes
Document sanitizer + detectors	`agents/reader/schema.py` , `agents/reader/anomalies.py`	U-SANIT, U-DET	passes

---

6. Measured security results (replay, static benchmark)

From `eval/reports/latest.json` : 96 attack cases (24 families × 4). Engine-level attack success rate — **B0 no-policy baseline 69.8% · B1 prompt-only baseline 40.6% · Mandate 0%** (95% Wilson CI [0, 0.039]). False-block result: 0 of 4 benign cases blocked. Failures are listed in the report; none occurred for Mandate on this run.

Scope (important). This is an **engine-level** measurement: each case supplies a labelled `ReaderOutput` and anomaly flags, so the result shows the deterministic engine authorises no violating payment *given correct fields*. It does **not** measure document parsing, OCR or hidden-text detection, and it is **not** a live-LLM end-to-end evaluation. B0/B1 are structural baselines (deterministic doubles in replay mode), not hosted-model measurements. The document-level detector path is exercised separately by `tests/test_prompt_injection_pdfs.py` (37 tests). This is a **static** benchmark on synthetic documents; adaptive attackers may do better.

---

7. Known-risk list (from docs/bugs.md §5)

The full register lives in `bugs.md` §5. Headline residual risks: - **R-002** a mis-extraction that still satisfies the mandate and registry (e.g. a wrong amount under the cap) can lead to a wrong-but-permitted payment. The LLM has no authority to execute, but it can influence what is proposed. - **R-003** prompt injection: the benchmark is static; adaptive attacks may do better. - **R-008** social engineering of the guardian — a deceived human can still approve. - **R-021** a signature proves a record was not altered, **not** that the institution is genuine. - **R-024** a bug in the executor or compromise of the engine key defeats the execution boundary. - **R-025** benchmark overfitting risk — mitigated by a frozen held-out split, run once.

---

8. Limitations

Sandbox only (no real money/banks/KYC); synthetic data; unverified Bangladesh rule pack (`data/rulepacks`); attestation is a prototype; registry governance is out of scope.

---

9. Reproduction

```
make setup && make test && make lint && make eval
```

Then open the generated `eval/reports/latest.html`, or `GET /api/v1/eval/report`.